

Sistema de Recomendação de Vídeo via Aprendizagem por Reforço

Grupo 4 - Diogo Azevedo, Eduardo Vilaça, Martim Ferreira, Oleksii Tantsura

UC Aprendizagem por Reforço, 2025/2026

<https://github.com/013ksii/AR>

Resumo Este artigo apresenta o desenvolvimento e a avaliação de um sistema de recomendação sequencial inspirado em feeds de vídeo de scroll infinito. O problema é formulado como um Processo de Decisão de Markov, no qual o estado representa o utilizador e o histórico recente da sessão, a ação corresponde ao subreddit recomendado e a recompensa aproxima o envolvimento implícito do utilizador. O sistema foi construído a partir de dados de interações do Reddit e integra uma arquitetura modular composta por pré-processamento, ambiente compatível com Gymnasium, baselines clássicos, algoritmos de multi-armed bandits, agentes de Aprendizagem por Reforço, uma API em FastAPI e uma interface de demonstração em React/Vite. A recompensa usada na interação online foi alinhada com a função de recompensa do ambiente de treino, garantindo consistência entre avaliação offline e demonstração interativa. Foram comparados métodos não-RL, epsilon-greedy, UCB, Thompson Sampling, Q-Learning, DQN e REINFORCE. Os resultados indicam que o REINFORCE obtém a maior recompensa acumulada média, com 22,71 pontos por episódio, enquanto o Thompson Sampling apresenta o melhor NDCG@10, com 0,475.

Palavras-chave: Aprendizagem por Reforço; Sistemas de Recomendação; Reddit; Feed Sequencial; REINFORCE; Bandits

1 Introdução

As plataformas digitais de consumo sequencial, como feeds de vídeo curto e redes sociais baseadas em scroll infinito, impõem requisitos distintos dos sistemas de recomendação tradicionais. Nestes contextos, cada recomendação influencia a continuidade da sessão, o grau de exploração de novos conteúdos e a percepção de variedade por parte do utilizador. Assim, a recomendação deixa de ser apenas uma tarefa de ordenação pontual e passa a constituir um problema de decisão sequencial.

A motivação deste trabalho resulta da necessidade de estudar métodos capazes de otimizar o envolvimento acumulado ao longo de uma sessão, preservando simultaneamente diversidade e cobertura do catálogo. A Aprendizagem por Reforço é particularmente adequada a esta formulação, pois permite modelar a interação entre agente e ambiente através de estados, ações, recompensas e transições. O agente aprende uma política de recomendação que procura maximizar o retorno esperado:

$$G_t = E \left[\sum_{k=0}^{T-t} d^k r_{t+k} \right], \quad (1)$$

em que T representa o comprimento da sessão, r_t a recompensa observada no instante t e d o fator de desconto.

O problema abordado consiste em recomendar subreddits a utilizadores, com base em interações históricas agregadas, simulando uma sessão de scroll com comprimento fixo. Os objetivos principais são: construir um ambiente de simulação realista a partir de dados públicos do Reddit; implementar e comparar baselines, bandits e agentes de Aprendizagem por Reforço; analisar a qualidade das recomendações através de métricas de ranking e de comportamento do catálogo; e disponibilizar uma interface de demonstração que represente a dinâmica de um feed interativo.

2 Estado da Arte

Os sistemas de recomendação clássicos recorrem frequentemente a popularidade global, perfis personalizados, filtragem colaborativa ou fatorização matricial. Estas abordagens são eficazes quando o objetivo é prever preferências estáticas ou produzir rankings pontuais, mas apresentam limitações em cenários onde a ordem das recomendações altera o estado da sessão e o comportamento subsequente do utilizador.

A Aprendizagem por Reforço fornece um enquadramento formal para recomendação sequencial. Afsar et al. [1] identificam três famílias predominantes em sistemas de recomendação baseados em RL: métodos baseados em modelo, métodos livres de modelo e abordagens híbridas. Nestes trabalhos, o problema é normalmente formulado como um Processo de Decisão de Markov, no qual o histórico de interações do utilizador é usado para construir o estado.

Table 1. Resumo dos dados considerados após filtragem.

Característica	Interações	Metadados Pós-filtragem	
Linhas totais	1 738 737	57 856	841 519
Utilizadores distintos	37 845	–	28 480 treino
Subreddits distintos	69 490	57 856	1000
Cobertura por metadados	83,3 por cento	–	–
Subreddits adultos excluídos	–	6238	–

Zhao et al. [2] exploram Deep Reinforcement Learning para recomendações page-wise, defendendo que a recomendação de listas ou sequências de itens permite modelar melhor a navegação real do utilizador. Chen et al. [3] aplicam REINFORCE à recomendação top-K com feedback implícito, mostrando que gradientes de política podem ser utilizados para otimizar diretamente decisões de recomendação em ambientes de grande escala.

Os algoritmos de multi-armed bandits, como UCB e Thompson Sampling, são também relevantes em recomendação, uma vez que formalizam o compromisso entre exploração de itens desconhecidos e exploração de itens já estimados como promissores [5, 6]. No entanto, estes métodos tendem a tratar cada ação de modo mais independente, não capturando plenamente a evolução do estado da sessão, ao contrário de métodos RL completos.

3 Metodologia

3.1 Dados e pré-processamento

Foram utilizados três ficheiros de dados. O ficheiro `reddit_user_data_count.csv` contém interações agregadas entre utilizadores e subreddits, representadas por `user`, `subreddit` e `count`. O ficheiro `subreddit_info.csv` contém metadados, incluindo número de subscritores, marcação `over18` e descrição pública. O ficheiro `reddit_data.csv` contém registos adicionais de publicações.

O pré-processamento remove subreddits sem metadados, exclui subreddits marcados como adultos, remove utilizadores com menos de cinco interações e subreddits com menos de dez interações, codifica utilizadores e subreddits em identificadores inteiros e limita o espaço de ações aos 1000 subreddits mais frequentes. A recompensa de base é calculada por transformação logarítmica:

$$r_{base}(u, s) = \log(1 + \text{count}(u, s)), \quad (2)$$

reduzindo o impacto de valores extremos. A divisão treino/teste é feita por utilizador, com 80 por cento dos utilizadores para treino e 20 por cento para teste.

3.2 Ambiente de simulação

O ambiente foi implementado como uma classe compatível com a interface Gymnasium. A formulação segue um Processo de Decisão de Markov $M = (S, A, P, R, d)$. O estado agrega o identificador do utilizador, os últimos subreddits recomendados e as recompensas recentes:

$$s_t = [u, a_{t-1}, \dots, a_{t-H}, r_{t-1}, \dots, r_{t-H}], \quad (3)$$

com janela histórica $H = 5$. O espaço de ações é discreto e contém 1000 subreddits. Cada episódio representa uma sessão de 20 recomendações.

A recompensa final do ambiente aplica reward shaping à recompensa de base. Em particular, a recompensa logarítmica é elevada ao quadrado, sendo depois ajustada por uma penalização de repetição e por um bónus de diversidade:

$$r_t = r_{base}(u, a_t)^2 - LI(a_t \in S_t) + BI(a_t \notin S_t), \quad (4)$$

em que S_t é o conjunto de subreddits já recomendados na sessão, $L = 0,5$ corresponde à penalização por repetição e $B = 0,1$ corresponde ao bónus de diversidade. A função $I(\cdot)$ representa uma variável indicadora, que assume valor 1 quando a condição é verdadeira e 0 caso contrário. Esta formulação valoriza interações fortes, reduz o efeito de repetição excessiva e incentiva a exploração controlada de novos subreddits.

4 Implementação e Arquitetura

O sistema desenvolvido encontra-se organizado de forma modular. A arquitetura contém uma componente offline de aprendizagem e avaliação, implementada em Python, uma API em FastAPI para servir recomendações e uma componente de demonstração interativa, implementada em React/Vite. A Tabela 2 resume os principais módulos.

A arquitetura segue um fluxo sequencial. Inicialmente, os dados brutos do Reddit são filtrados e convertidos para um formato adequado ao treino. Em seguida, o ambiente simulado recebe os dados processados e fornece uma interface comum para baselines, bandits e agentes de Aprendizagem por Reforço. Após o treino, os modelos são avaliados através de métricas de ranking e de comportamento do catálogo. Por fim, a API e o frontend permitem demonstrar a recomendação em modo interativo.

4.1 Algoritmos implementados

Foram implementados quatro baselines não-RL: recomendação aleatória uniforme, popularidade global, popularidade personalizada e filtragem colaborativa baseada em utilizadores. A filtragem colaborativa constrói uma matriz utilizador-subreddit e calcula similaridade de cosseno entre utilizadores, recomendando itens associados aos vizinhos mais próximos.

Table 2. Arquitetura funcional do sistema desenvolvido.

Módulo	Função principal
src/preprocessing	Limpeza, filtragem, codificação e geração de ficheiros processados.
src/environment	Ambiente de recomendação compatível com Gymnasium e função de recompensa.
src/baselines	Random, popularidade global, popularidade personalizada e filtragem colaborativa.
src/agents	Bandits, Q-Learning, DQN e REINFORCE.
src/training	Ciclos de treino, gravação de modelos e curvas de aprendizagem.
src/evaluation	Métricas de avaliação e geração de resultados.
src/api	API FastAPI para recomendações, interações e conteúdos simulados do Reddit.
frontend	Interface React/Vite para simulação visual do feed e recolha de feedback.

Os métodos de multi-armed bandits incluem epsilon-greedy, UCB e Thompson Sampling. O algoritmo epsilon-greedy mantém estimativas por utilizador e utiliza um prior global para novos utilizadores. O UCB usa estimativas por utilizador e restringe a exploração inicial aos itens mais promissores segundo o prior global. O Thompson Sampling utiliza priors Beta e atualiza a noção de sucesso através de um limiar adaptativo baseado na mediana das recompensas recentes.

O Q-Learning usa uma tabela $Q(s, a)$, com estado discretizado pelo par composto pelo utilizador e pelo último subreddit recomendado. O DQN substitui a tabela por uma rede neuronal com embeddings de utilizador e de subreddits, replay buffer, rede-alvo e política epsilon-greedy. O REINFORCE utiliza uma rede de política que produz logits sobre o espaço de subreddits e atualiza os pesos por gradiente de política com retornos descontados e normalizados:

$$J(\theta) \approx \frac{1}{T} \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t) G_t. \quad (5)$$

4.2 API, frontend e alinhamento da recompensa

A componente de demonstração interativa integra uma API em FastAPI com uma interface React/Vite. A API expõe operações para gerar recomendações para um utilizador, registar interações e consultar conteúdos associados a um subreddit. O frontend apresenta os itens recomendados sob a forma de feed, recolhe sinais de interação e envia o feedback para o backend.

Para garantir coerência entre a aprendizagem offline e a interação online, a recompensa usada pelo frontend foi alinhada com a função de recompensa usada no treino. O frontend estima apenas uma recompensa de base, calculada a partir de sinais de interação, como tempo de visualização e feedback explícito. Essa

recompensa de base segue a mesma escala logarítmica usada no pré-processamento, sendo definida por $\hat{r}_{base} = \log(1 + v)$, em que v representa uma aproximação do nível de envolvimento observado na interface. Esta recompensa de base é enviada para a API, que aplica a mesma transformação usada no ambiente de treino: $\hat{r}_{online} = \hat{r}_{base}^2 - LI(a_t \in S_t) + BI(a_t \notin S_t)$.

Desta forma, a política recebe na demonstração online recompensas compatíveis com as que foram usadas durante o treino e a avaliação offline. Esta decisão reduz discrepâncias entre o comportamento aprendido pelo agente e o sinal de feedback usado durante a execução interativa.

Para a demonstração visual, a API inclui ainda um proxy local para obter conteúdos associados aos subreddits recomendados. Esta componente é independente do treino e da avaliação dos agentes, que utilizam dados previamente processados. Devido às restrições atuais de acesso ao Reddit e à necessidade de autenticação em alguns pedidos, o proxy pode requerer uma sessão autenticada local, fornecida através de variável de ambiente. Por razões de segurança e reprodutibilidade, credenciais, cookies e tokens não são incluídos no repositório. Caso o acesso em tempo real não esteja disponível, a demonstração pode recorrer a conteúdos simulados ou previamente guardados, sem afetar os resultados experimentais apresentados.

A Figura 1 apresenta a documentação Swagger/OpenAPI da API desenvolvida, onde são expostos os endpoints de recomendação, registo de interações e obtenção de conteúdos associados a subreddits. A Figura 2 ilustra a interface de demonstração, composta por um feed de recomendações e por um painel lateral com o estado de memória do agente, incluindo ações recentes e recompensas observadas.

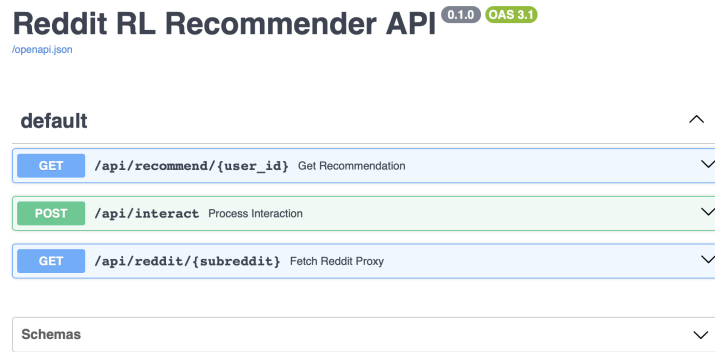
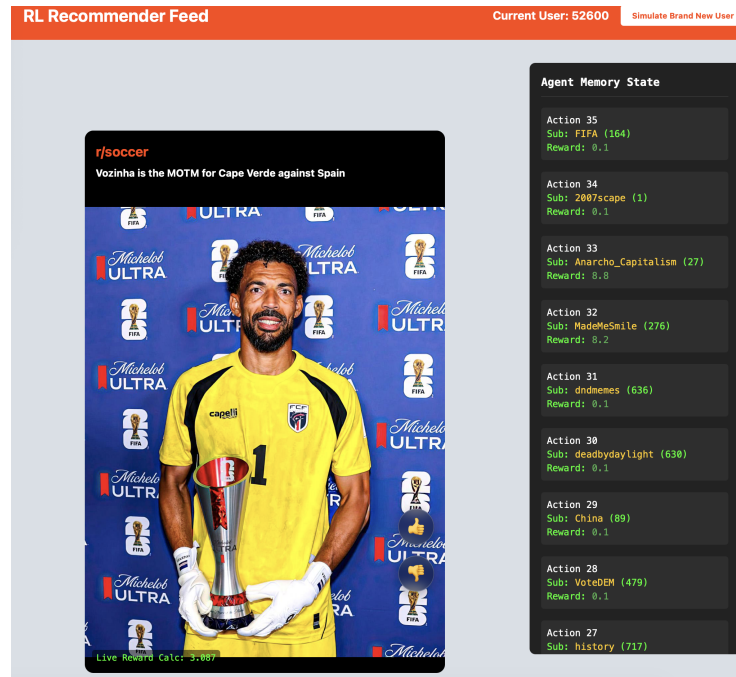


Fig. 1. Documentação Swagger/OpenAPI da API desenvolvida, com endpoints de recomendação, interação e proxy Reddit.

Table 3. Métricas de avaliação utilizadas.

Métrica	Interpretação
Reward	Recompensa acumulada média ao longo do episódio.
HR@10	Proporção de episódios com pelo menos um item relevante no top-10.
NDCG@10	Qualidade da ordenação das recomendações no top-10.
Diversidade	Fração de subreddits distintos recomendados no episódio.
Cobertura	Fração do catálogo total efetivamente recomendada.
Novidade	Grau de recomendação de subreddits menos populares.

**Fig. 2.** Interface de demonstração do sistema, com feed de recomendações, botões de interação, cálculo de recompensa e memória recente do agente.

5 Avaliação Experimental

Todos os agentes de aprendizagem foram treinados durante 5000 episódios. A avaliação foi realizada em 200 episódios sobre utilizadores de teste não vistos durante o treino. As métricas usadas foram recompensa acumulada média por episódio, Hit Rate@10, NDCG@10, diversidade intra-lista, cobertura do catálogo e novidade.

Table 4. Resultados de avaliação em 200 episódios de teste.

Agente	Reward	HR@10	NDCG@10	Div.	Cob.	Nov.
Random	2,73	0,110	0,011	0,993	0,973	0,975
GlobalPop	2,68	0,075	0,008	1,000	1,000	0,976
PersonalPop	9,55	0,645	0,183	1,000	0,040	0,737
CollabFilter	8,79	0,680	0,186	1,000	0,040	0,791
epsilon-greedy	3,73	0,200	0,178	0,150	0,322	0,646
UCB	-4,44	0,085	0,087	0,050	0,001	0,817
Thompson	20,09	0,465	0,475	0,095	0,139	0,044
Q-Learning	2,63	0,115	0,015	0,992	0,982	0,976
DQN	2,78	0,070	0,007	0,992	0,982	0,976
REINFORCE	22,71	0,485	0,285	0,481	0,445	0,437

Table 5. Representação textual da recompensa acumulada média.

Agente	Reward	Barra relativa
REINFORCE	22,71	
Thompson	20,09	
PersonalPop	9,55	
CollabFilter	8,79	
epsilon-greedy	3,73	
DQN	2,78	
Random	2,73	
Q-Learning	2,63	
GlobalPop	2,68	
UCB	-4,44	valor negativo

6 Resultados e Discussão

A Tabela 4 apresenta os resultados principais. O REINFORCE obteve a maior recompensa acumulada média, enquanto o Thompson Sampling obteve o melhor NDCG@10.

O REINFORCE alcançou a melhor recompensa acumulada média, com 22,71 pontos por episódio, seguido pelo Thompson Sampling, com 20,09. Ambos superaram de forma clara os baselines não-RL. Este resultado sugere que a otimização de políticas sequenciais é vantajosa quando a qualidade da sessão depende das decisões anteriores e não apenas da relevância pontual de cada item.

O desempenho do Q-Learning e do DQN ficou próximo do Random. No Q-Learning, a discretização do estado é demasiado simples para capturar preferências complexas. No DQN, a aprendizagem de embeddings a partir do zero num espaço de 1000 ações exige mais dados, mais episódios ou pré-treino. Esta limitação é particularmente relevante em recomendação, onde representações semânticas de itens e utilizadores podem determinar a capacidade de generalização.

Em Hit Rate@10, o CollabFilter e o PersonalPop obtiveram os melhores valores, respetivamente 0,680 e 0,645. Esta vantagem decorre do acesso direto ao perfil histórico completo do utilizador. Contudo, estes métodos não modelam explicitamente o efeito das recomendações ao longo de uma sessão. O Thompson

Table 6. Representação textual do NDCG@10.

Agente	NDCG@10	Barra relativa
Thompson	0,475	
REINFORCE	0,285	
CollabFilter	0,186	
PersonalPop	0,183	
epsilon-greedy	0,178	
UCB	0,087	
Q-Learning	0,015	
Random	0,011	
GlobalPop	0,008	
DQN	0,007	

Sampling apresentou o melhor NDCG@10, com 0,475, indicando que tende a colocar os itens relevantes nas primeiras posições. O REINFORCE obteve menor NDCG@10, mas maior recompensa acumulada, evidenciando que a política aprendida privilegia a qualidade da trajetória completa e não apenas a ordenação inicial.

Os resultados mostram também um compromisso entre personalização e diversidade. PersonalPop e CollabFilter atingem diversidade intra-lista elevada, mas cobertura global muito reduzida, concentrando-se num subconjunto pequeno do catálogo. Thompson Sampling apresenta forte desempenho em NDCG@10, mas baixa diversidade e novidade, sinalizando risco de concentração em itens muito recompensadores. O REINFORCE apresenta uma solução mais equilibrada, com diversidade de 0,481 e cobertura de 0,445. Esta combinação é relevante para aplicações reais, pois uma política útil deve maximizar envolvimento sem reduzir excessivamente a descoberta de conteúdos.

7 Conclusões

Este trabalho demonstrou que a Aprendizagem por Reforço é uma abordagem adequada para modelar recomendação sequencial em feeds de scroll infinito. A formulação como Processo de Decisão de Markov permitiu representar o utilizador, o histórico recente e a recompensa acumulada numa estrutura unificada. Entre os métodos avaliados, o REINFORCE destacou-se em recompensa acumulada, enquanto o Thompson Sampling apresentou a melhor qualidade de ordenação segundo o NDCG@10.

A implementação modular permitiu comparar diferentes famílias de algoritmos sob o mesmo ambiente e com as mesmas métricas. A API em FastAPI e a interface React/Vite reforçam a dimensão aplicada do projeto, mostrando como uma política de recomendação pode ser integrada numa experiência interativa de feed, com recolha contínua de sinais de feedback. O alinhamento da função de recompensa entre treino, avaliação offline e demonstração online constitui um

aspecto importante da implementação, uma vez que reduz discrepâncias entre o comportamento aprendido e o sinal observado durante a execução.

As principais limitações são a natureza simulada do ambiente, a ausência de uma avaliação online com utilizadores reais, a aproximação da recompensa a partir de contagens históricas e a escala ainda reduzida do espaço de ações face a plataformas industriais. Como trabalho futuro, propõe-se explorar embeddings semânticos dos subreddits com modelos de linguagem, pré-treino de representações de utilizadores e itens, avaliação temporal com dados datados e integração completa entre o agente treinado, a API local e o frontend de demonstração.

Disponibilidade de Código e Anexos

O código-fonte, os dados processados, os resultados, os notebooks, a apresentação, os prints da aplicação e o relatório final encontram-se disponíveis no repositório público do grupo: <https://github.com/013ksii/AR>. Os anexos complementares poderão ser disponibilizados no mesmo repositório, de forma a manter o relatório principal dentro do limite de dez páginas.

References

1. Afsar, M.M., Crump, T., Far, B.H.: Reinforcement Learning Based Recommender Systems: A Survey. *ACM Computing Surveys* 55(7), 1–38 (2022). <https://doi.org/10.1145/3543846>
2. Zhao, X., Xia, L., Zhang, L., Ding, Z., Yin, D., Tang, J.: Deep Reinforcement Learning for Page-wise Recommendations. In: *Proceedings of the 12th ACM Conference on Recommender Systems*, pp. 95–103. ACM (2018)
3. Chen, M., Beutel, A., Covington, P., Jain, S., Belletti, F., Chi, E.H.: Top-K Off-Policy Correction for a REINFORCE Recommender System. In: *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*, pp. 456–464. ACM (2019)
4. Sutton, R.S., Barto, A.G.: *Reinforcement Learning: An Introduction*. 2nd edn. MIT Press, Cambridge (2018)
5. Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 235–256 (2002)
6. Chapelle, O., Li, L.: An Empirical Evaluation of Thompson Sampling. In: *Advances in Neural Information Processing Systems* 24, pp. 2249–2257 (2011)
7. Koren, Y., Bell, R., Volinsky, C.: Matrix Factorization Techniques for Recommender Systems. *Computer* 42(8), 30–37 (2009)